# Module3

# Convolutional Neural Network(CNN) or ConvNet

- Convolutional Neural Network(CNN) or ConvNet, is a class of neural networks.
- It specializes in processing data that has a grid-like topology, such as an image.
- A digital image is a binary representation of visual data. It contains a series of pixels arranged in a grid-like fashion that contains **pixel values** to denote how bright and what color each pixel should be.

| | | | | |
|---|---|---|---|---|
| -1 | 1 | 1 | 1 | -1 |
| -1 | 1 | -1 | 1 | -1 |
| -1 | 1 | 1 | 1 | -1 |
| -1 | -1 | -1 | 1 | -1 |
| -1 | -1 | -1 | 1 | -1 |
| -1 | -1 | 1 | -1 | -1 |
| -1 | 1 | -1 | -1 | -1 |



# Location shifted



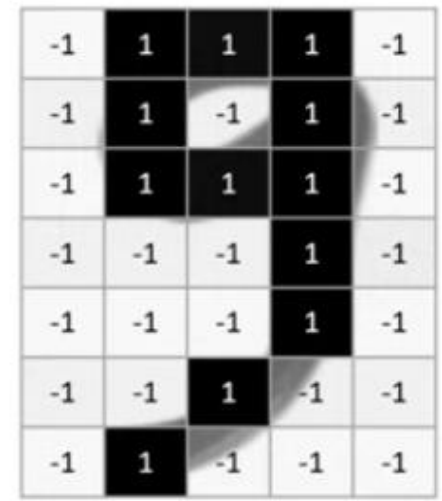
# Variation 1


-1 -1 1 -1 -1
-1 1 -1 1 -1
-1 1 1 -1 -1
-1 -1 1 -1 -1
-1 -1 1 -1 -1
-1 -1 1 -1 -1
-1 -1 1 -1 -1

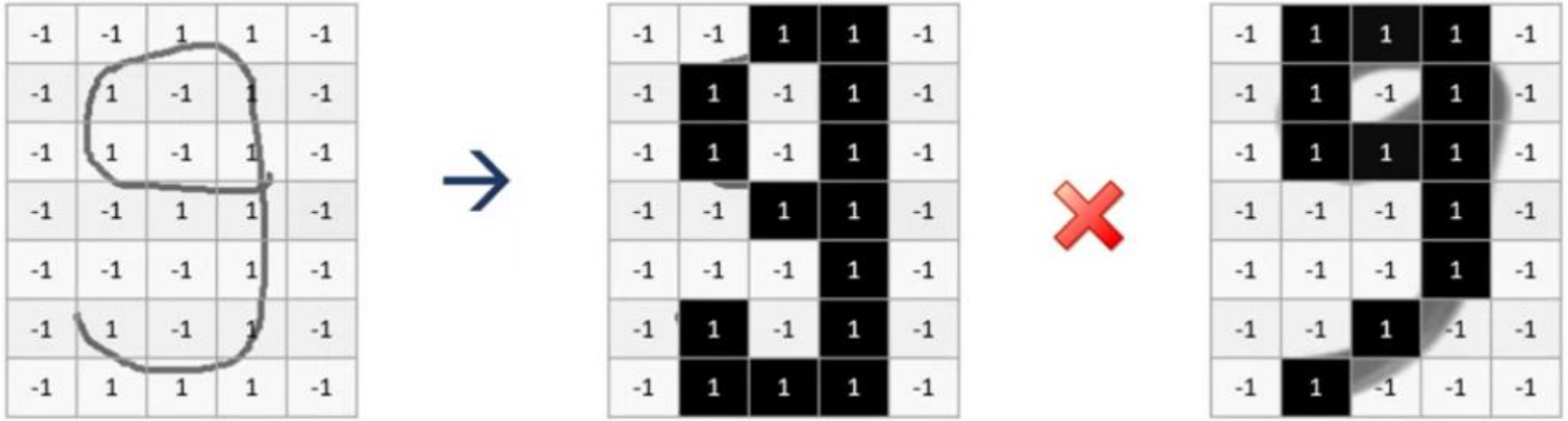



-1 -1 1 -1 -1
-1 1 -1 1 -1
-1 1 1 -1 -1
-1 -1 1 -1 -1
-1 -1 1 -1 -1
-1 -1 1 -1 -1
-1 -1 1 -1 -1




-1 1 1 1 -1
-1 1 -1 1 -1
-1 1 1 1 -1
-1 -1 -1 1 -1
-1 -1 -1 1 -1
-1 -1 1 -1 -1
-1 1 -1 -1 -1

# Variation 2


-1 -1 1 1 -1
-1 1 -1 1 -1
-1 1 -1 1 -1
-1 -1 1 1 -1
-1 -1 -1 1 -1
-1 1 -1 1 -1
-1 1 1 1 -1
-1 -1 1 1 -1
-1 1 -1 1 -1
-1 1 -1 1 -1
-1 -1 1 1 -1
-1 -1 -1 1 -1
-1 1 -1 1 -1
-1 1 1 1 -1
-1 1 1 1 -1
-1 1 -1 1 -1
-1 1 1 1 -1
-1 -1 -1 1 -1
-1 -1 -1 1 -1
-1 -1 1 -1 -1
-1 1 -1 -1 -1

# Images having high dimensions



Image size = 1920 x 1080 X 3

First layer neurons = 1920 x 1080 X 3 ~ 6 million

Hidden layer neurons = ~ 4 million

Weights between input and hidden layer = 6 mil * 4 mil
= 24 million

# Dis advantages of ANN for Image Classification

1. Too much Computation
2. Treats local pixels same as pixels far apart
3. Sensitive to location of an object in an image.

- The human brain processes  huge amount of information when we see an image. Each neuron works in its own receptive field and is connected to other neurons in a way that they cover the entire visual field. Each neuron responds to stimuli only in the restricted region of the visual field called the **receptive field** in the biological vision system.
- Each neuron in a CNN also processes data only in its receptive field.The layers are arranged in such a way so that they detect simpler patterns first (lines, curves, etc.) and then more complex patterns (faces, objects, etc.). By using a CNN, one can enable computers to see.
- Mainly two activities are carried out in CNN

- Feature engineering( extracting important features from an image)  &
- Classification

# Convolutional Neural Network Architecture

- A CNN typically has three layers:

✓ a convolutional layer

✓ a pooling layer, and

✓a fully connected layer.

# Convolution Layer

- The first layer of a Convolutional Neural Network is always a **Convolutional Layer.**
- The convolution layer is the core building block of the CNN. It carries the main portion of the network's computational load.
- Convolutional layers in a convolutional neural network summarize the presence of features in an input image.
- Convolutional layers apply a convolution operation to the input, passing the result to the next layer.

# Kernal or Filter

- It is also known as feature detector.
- It is used to extract features from an image
- Filter is a **matrix** that moves over the input data, performs the dot product with the sub region of input data and get the output as a matrix of dot product.
- Filter moves on the input data by the stride value.

# Convolution Operation

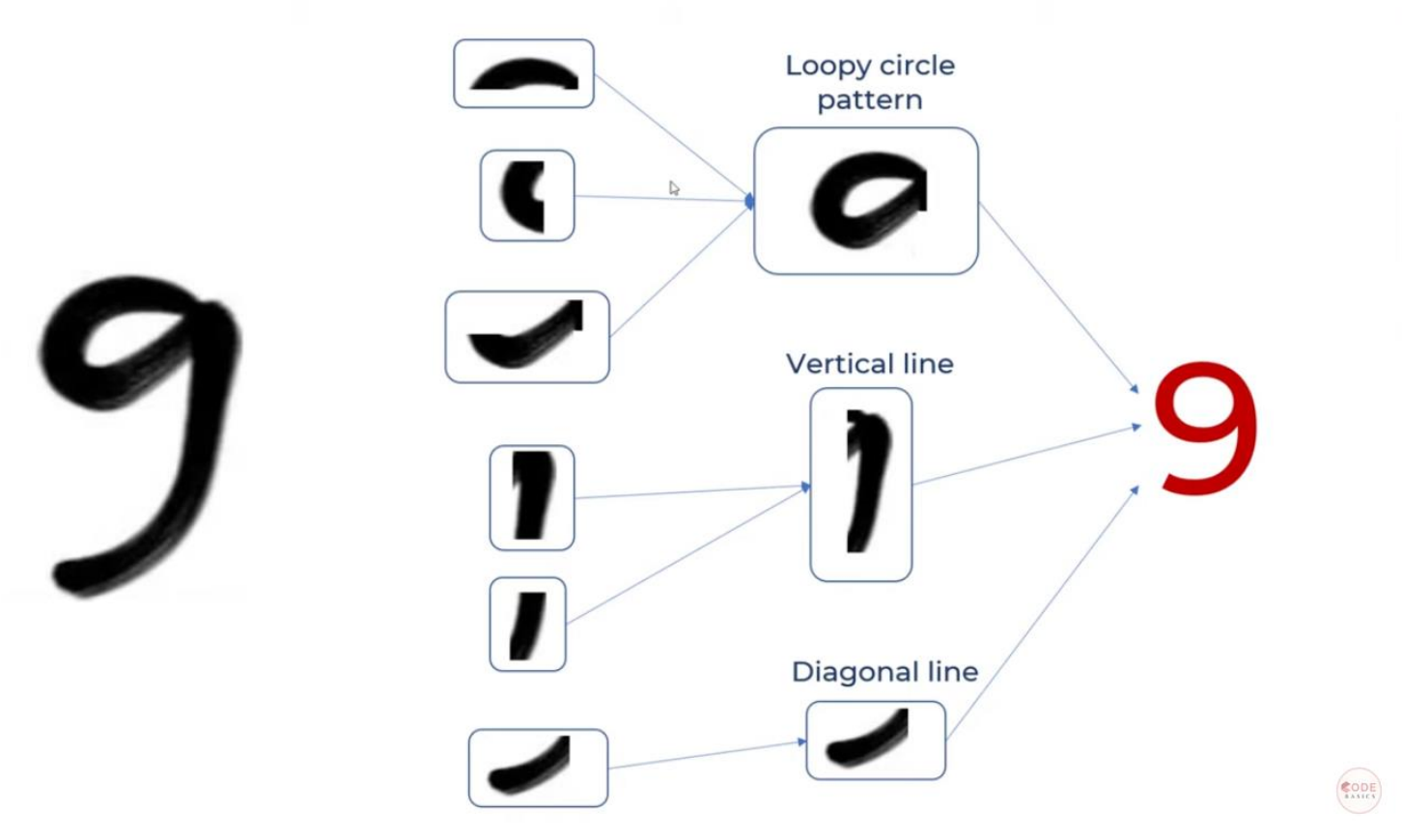
- A convolution converts all the pixels in its receptive field into a single value. For example, when we apply a convolution to an image, we will be decreasing the image size as well as bringing all the information in the field together into a single pixel. The final output of the convolutional layer is a vector. Based on the type of problem to be solved and the kind of features that are looked for, we can use different kinds of convolutions.

- The most common type of convolution that is used is the 2D convolution layer and is usually abbreviated as conv2D.
- A filter or a kernel in a conv2D layer “slides” over the 2D input data, performing an elementwise multiplication. As a result, it will be summing up the results into a single output pixel. The kernel will perform the same operation for every location it slides over, transforming a 2D matrix of features into a different 2D matrix of features. The image shrinks every time a convolution operation is performed.

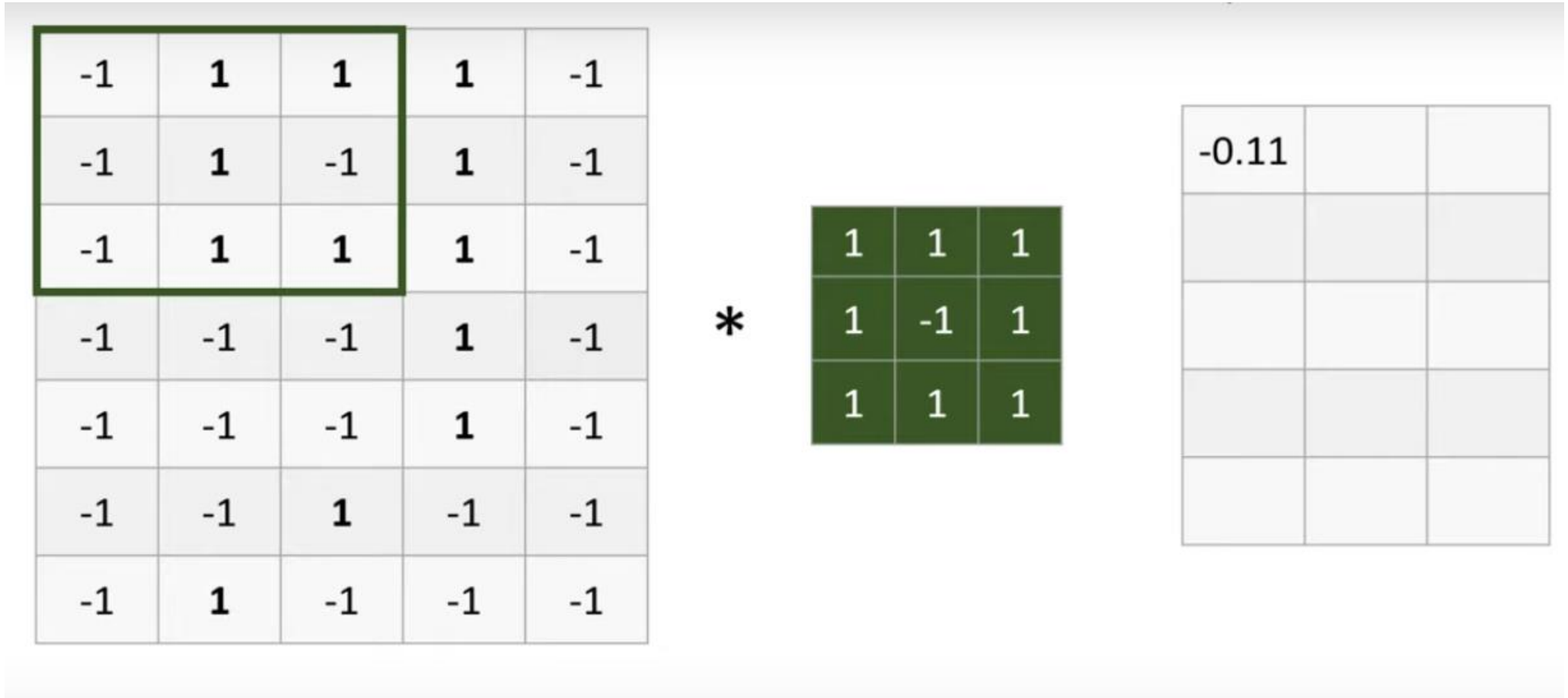
# Stride

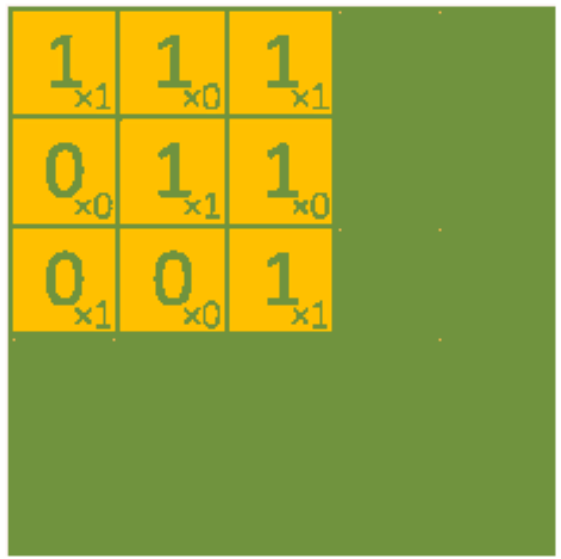
- The filter is moved across the image from left to right and top to bottom with a one pixel column change on the horizontal direction, then a one pixel row change on the vertical direction.
- The amount of movement between application of the filter to the input image is referred to as the Stride and it is always symmetrical in height and width dimensions.

-Stride values of two or greater is rare, a larger stride yields a smaller output.

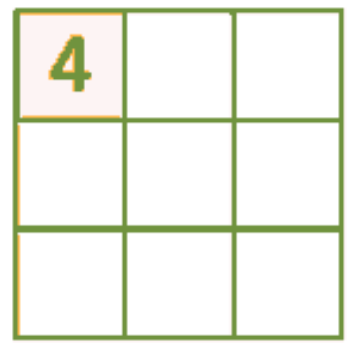

Loopy circle pattern
Vertical line
Diagonal line
9

-1+1+1-1-1-1-1+1+1 = -1 → -1/9 = -0.11



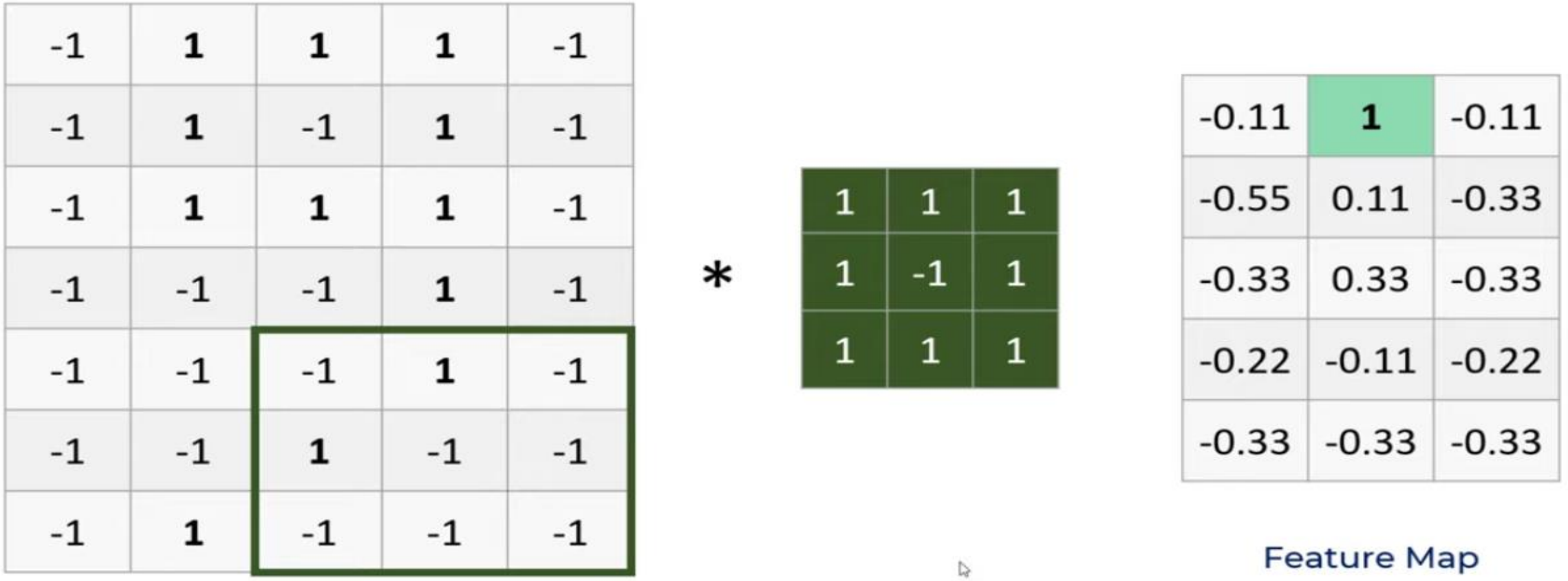



Image




Convolved Feature

# By doing Convolution operation, a feature map is created





*

Loopy pattern detector

| | | |
|---|---|---|
| 1 | 1 | 1 |
| 1 | -1 | 1 |
| 1 | 1 | 1 |

=

| | | |
|---|---|---|
| | 1 | |
| | | |
| | | |
| | | |
| | | |



*

Loopy pattern detector

| | | |
|---|---|---|
| 1 | 1 | 1 |
| 1 | -1 | 1 |
| 1 | 1 | 1 |

=

| | | |
|---|---|---|
| | | |
| | | |
| | | |
| | | |
| | 1 | |



*

Loopy pattern detector

| | | |
|---|---|---|
| 1 | 1 | 1 |
| 1 | -1 | 1 |
| 1 | 1 | 1 |

=

| | | |
|---|---|---|
| | 1 | |
| | | |
| | | |
| | | |
| | 1 | |



*

Loopy pattern detector

| | | |
|---|---|---|
| 1 | 1 | 1 |
| 1 | -1 | 1 |
| 1 | 1 | 1 |

=

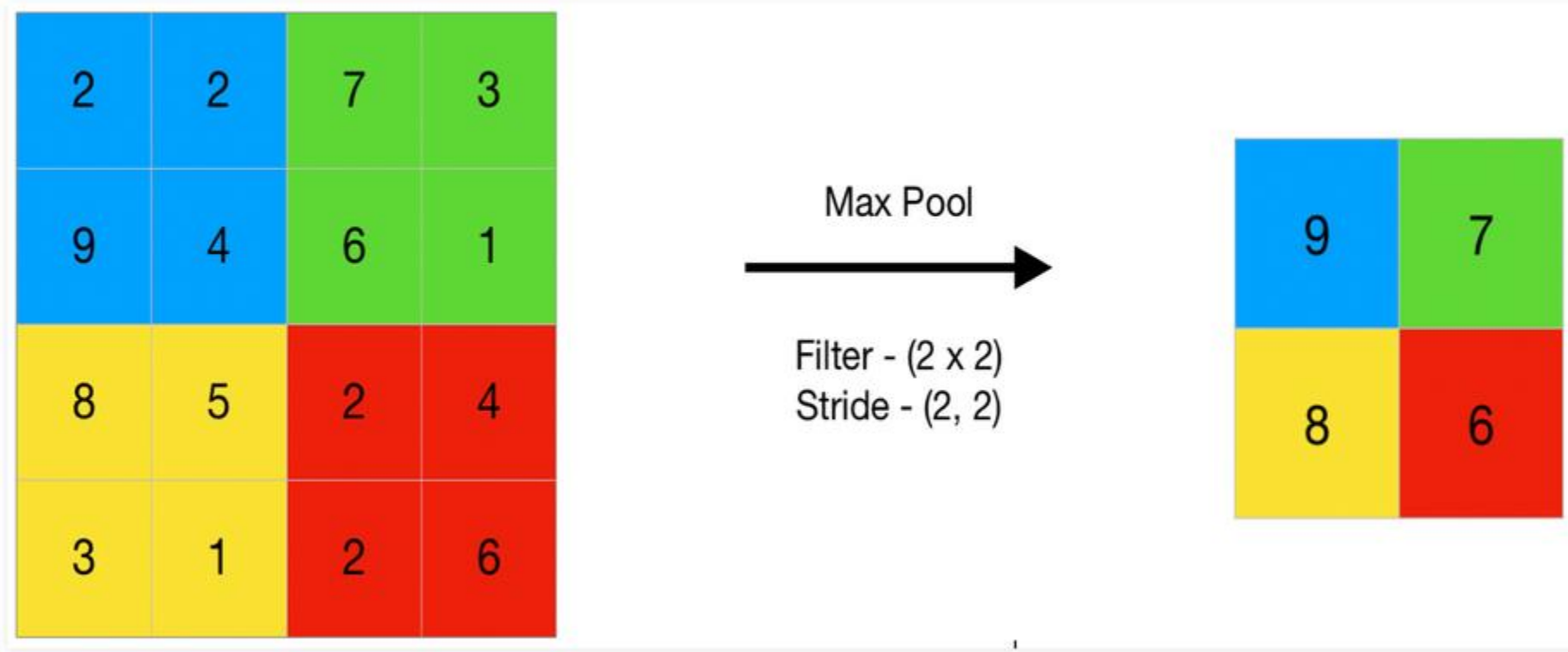
| | | | | |
|---|---|---|---|---|
| | 1 | | | |
| | | | | |
| | | | | |
| | | | | |
| | | | 1 | |

- During the forward pass, the filter slides across the height and width of the image-producing the image representation of that receptive region. This produces a two-dimensional representation of the image known as an activation map or feature map that gives the response of the kernel at each spatial position of the image. The sliding size of the filter is called a stride.
- After each convolution operation, a CNN applies a Rectified Linear Unit (ReLU) transformation to the feature map, introducing nonlinearity to the model.

The **number of filters** affects the depth of the output. For example, three distinct filters would yield three different feature maps, creating a depth of three.

# Pooling Layer

- Pooling layers are used to reduce the dimensions of the feature maps. Thus, it reduces the number of parameters to learn and the amount of computation performed in the network.
- The pooling layer summarises the features present in a region of the feature map generated by a convolution layer. So, further operations are performed on summarised features instead of precisely positioned features generated by the convolution layer. This makes the model more robust to variations in the position of the features in the input image. It **decreases the computational power required to process the data** through dimensionality reduction.
- Types of Pooling Layers:
- **Max Pooling**
- Max pooling is a pooling operation that selects the maximum element from the region of the feature map covered by the filter. Thus, the output after max-pooling layer would be a feature map containing the most prominent features of the previous feature map.


2 2 7 3
9 4 6 1
8 5 2 4
3 1 2 6
Max Pool
Filter - (2 x 2)
Stride - (2, 2)
9 7
8 6

# Average Pooling

Average pooling computes the average of the elements present in the region of feature map covered by the filter. Thus, while max pooling gives the most prominent feature in a particular patch of the feature map, average pooling gives the average of features present in a patch.



- Max Pooling performs as a **Noise Suppressant**.
- It discards the noisy activations altogether and also performs de-noising along with dimensionality reduction.
- Average Pooling simply performs dimensionality reduction as a noise suppressing mechanism.
- Hence can say that **Max Pooling performs better than Average Pooling**.

- **Global Pooling**
- Global pooling reduces each channel in the feature map to a single value. Thus, an $\mathbf{n_h \times n_w \times n_c}$ feature map is reduced to $\mathbf{1 \times 1 \times n_c}$ feature map. This is equivalent to using a filter of dimensions $\mathbf{n_h \times n_w}$ i.e. the dimensions of the feature map.

# Flattening

- Flattening operation takes the numbers **row by row** and put them into a single column or converts a 2 D array into a 1D array.
- This is done to input an image data into an artificial neural network for further processing.
- By flattening the pooled layer, we can get one huge vector of inputs for an artificial neural network.

# Fully Connected Layer

- Fully Connected Layer is simply a **feed forward neural networks**.
- Fully Connected Layers form the **last few layers in the network**. All neurons in a fully connected layer connect to all the neurons in the previous layer.
- After feature extraction, the data should be **classified into various classes**. This is done using a fully connected (FC) neural network.
- The input to the fully connected layer is the output from the final Pooling or Convolutional Layer, which is flattened and then fed into the fully connected layer.

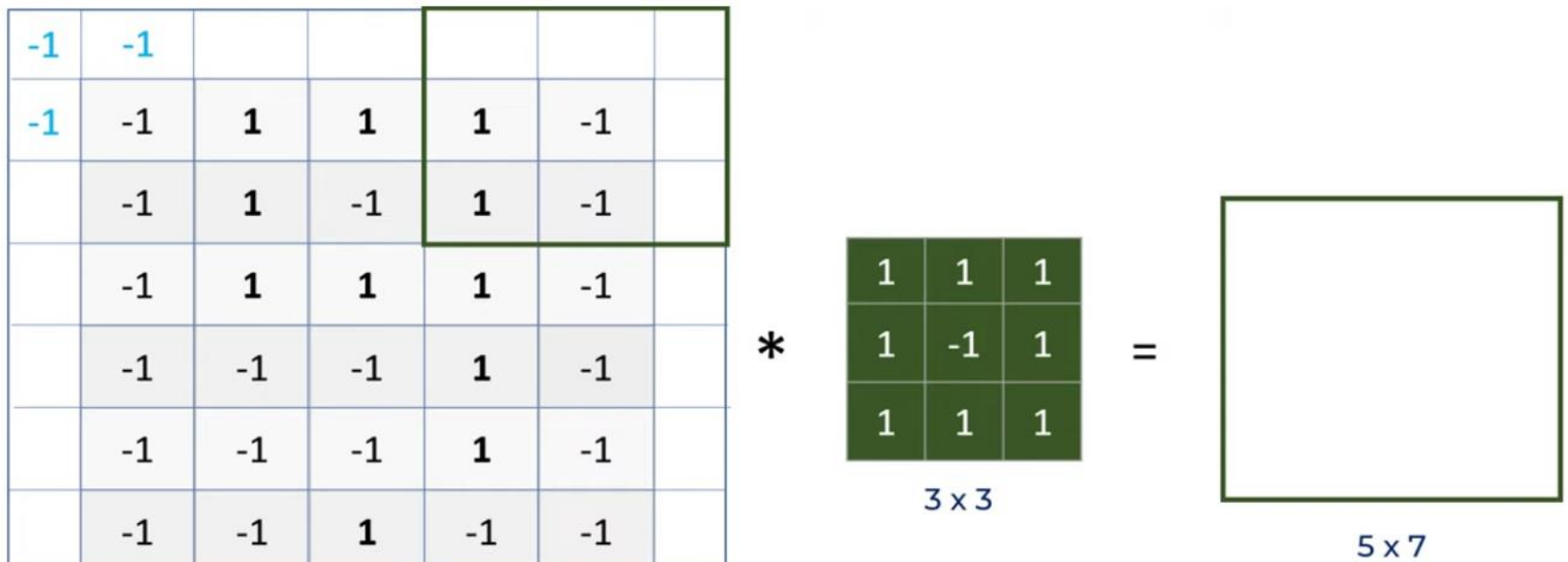
- The flattened output is fed to a feed-forward neural network and backpropagation applied to every iteration of training.
- Over a series of epochs, the model is able to distinguish between dominating and certain low-level features in images and classify them using any classification technique.

# Padding

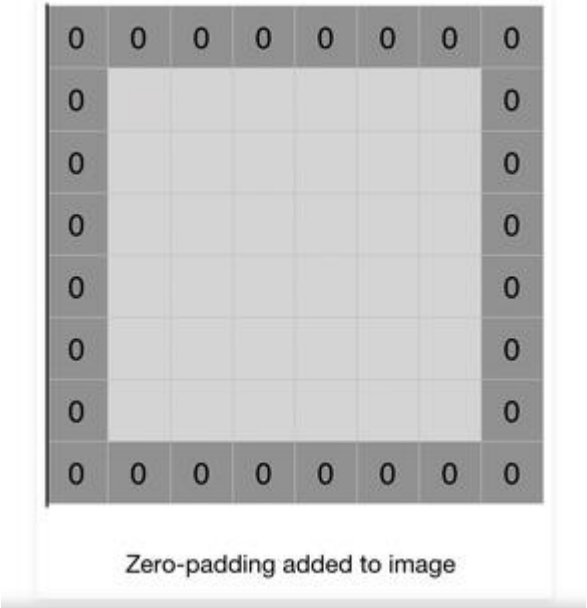
| | | | | |
|---|---|---|---|---|
| -1 | 1 | 1 | 1 | -1 |
| -1 | 1 | -1 | 1 | -1 |
| -1 | 1 | 1 | 1 | -1 |
| -1 | -1 | -1 | 1 | -1 |
| -1 | -1 | -1 | 1 | -1 |
| -1 | -1 | 1 | -1 | -1 |
| -1 | 1 | -1 | -1 | -1 |

| | | | | |
|---|---|---|---|---|
| -1 | 1 | 1 | 1 | -1 |
| -1 | 1 | -1 | 1 | -1 |
| -1 | 1 | 1 | 1 | -1 |
| -1 | -1 | -1 | 1 | -1 |
| -1 | -1 | -1 | 1 | -1 |
| -1 | -1 | 1 | -1 | -1 |
| -1 | 1 | -1 | -1 | -1 |

Corner Pixels do not contribute much in feature detection.


-1 -1
-1 -1 1 1 1 -1
-1 1 -1 1 -1
-1 1 1 1 -1
-1 -1 -1 1 -1
-1 -1 -1 1 -1
-1 -1 1 -1 -1
-1 1 -1 -1 -1
*
1 1 1
1 -1 1
1 1 1
=
7 x 9 (m x n)
Padding = 1
3 x 3
5 x 7

- Padding is simply a process of adding layers of zeros to an input image.
-



- Padding increases the contribution of the pixels at the border of the original image by bringing them into the middle of the padded image. Thus, information on the borders is preserved as well as the information in the middle of the image.
- **Valid Padding:** It implies no padding at all. The input image is left in its valid/unaltered shape.
- **Same Padding:** In this case, ‘p’ padding layers are added such that the output image has the same dimensions as the input image.

# Types of CNNs

- Transfer learning is the method that uses a neural network trained on a large and generalized dataset and being used for solving another problem. These neural networks are called Pre-trained networks.

- The basic requirement for transfer learning is the availability of pre-trained neural networks.For computer vision, following pre-trained CNNs are available.
- Some of them are :

**1.LeNet**

**2.AlexNet**

**3.VGGNet**

4.GoogLeNet

5.ResNet

6.ZFNet

# LeNet

- It is the first convolutional neural network architecture designed specifically for  recognizing numeric digits from handwritten or printed images.
- It can classify 2 dimensional images or gray scale images.
- Goal of Lenet is to recognize digits written in bank cheques.
- It was developed when no GPUs were available.
- It has a  simple and straightforward architecture. It is a 7-layer convolution neural network for image classification.

# LeNet -5 Architecture

- It has 2 convolution layers , 2 pooling layers and 2 fully connected layers and 1 output layer.
- In the output layer, a Softmax classifier classifies an image into a particular class based on probability scores.



| | | | | | | |
|---|---|---|---|---|---|---|
| 0 | 0 | 0 | 0 | 0 | .. | 0 |
| 0 | 87 | 240 | 210 | 24 | .. | 0 |
| 0 | 13 | 0 | 101 | 195 | .. | 0 |
| 0 | 35 | 167 | 99 | 210 | .. | 0 |
| 0 | 145 | 230 | 240 | 201 | .. | 140 |
| .. | .. | .. | .. | .. | .. | .. |
| 0 | 0 | 0 | 0 | 0 | .. | 0 |

28 by 28 grid

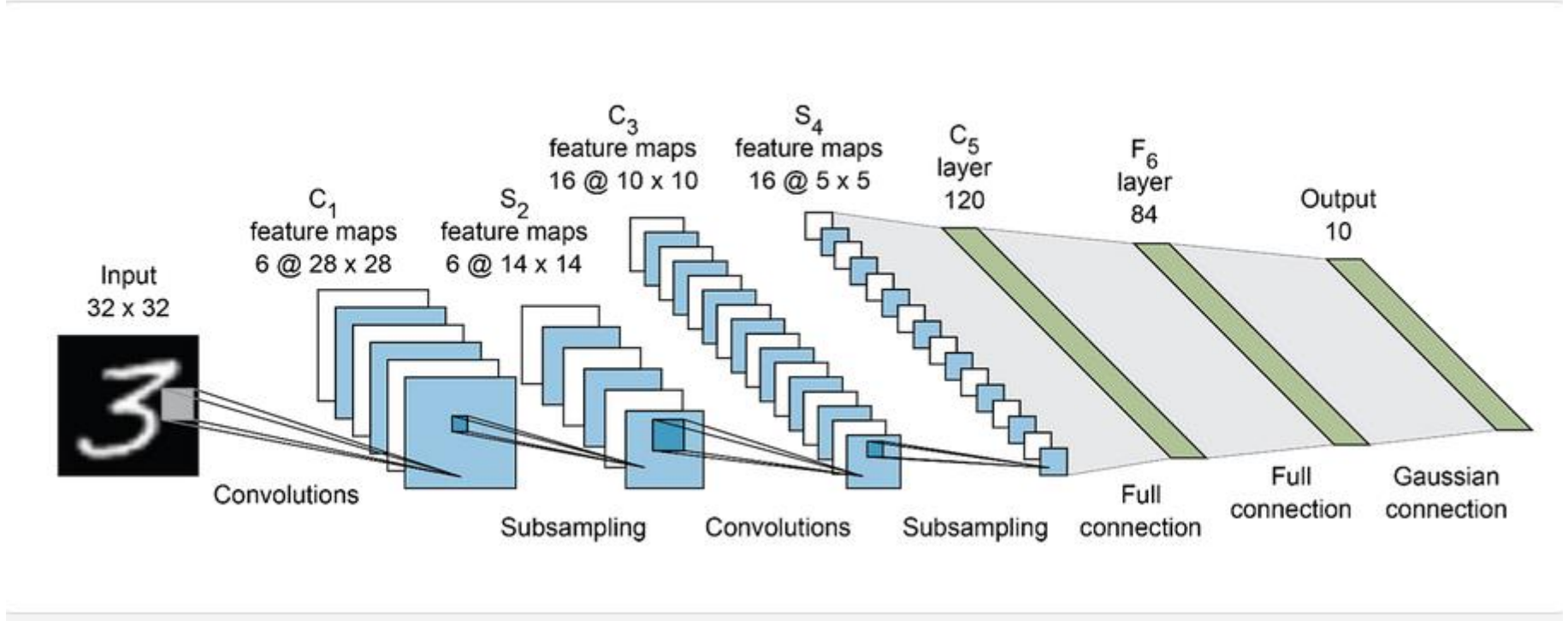
0
0
0
0
0
...
0
0
87
240
210
24
0
...
0

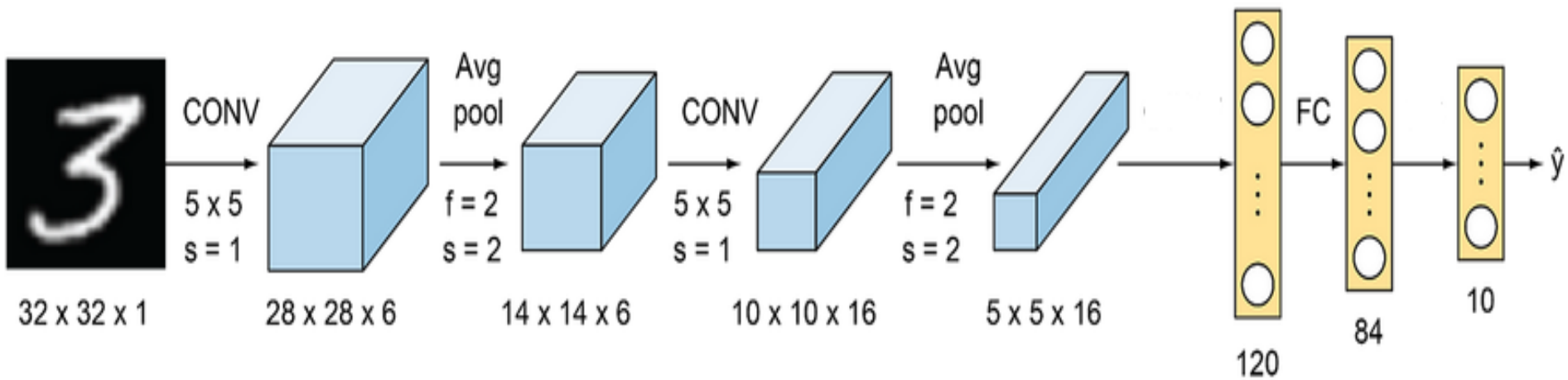
$x_1$
$x_2$
$x_3$
$x_{784}$

$\Sigma$ $\sigma$ → 0 0.29
$\Sigma$ $\sigma$ → 1 0.07
$\Sigma$ $\sigma$ → 2 **0.92**
$\Sigma$ $\sigma$ → 3 0.08
$\Sigma$ $\sigma$ → 4 0.001
$\Sigma$ $\sigma$ → 9 0.043

Activate Windows

# Lenet 5 Architecture




CONV
5 x 5
s = 1
Avg
pool
f = 2
s = 2
CONV
5 x 5
s = 1
Avg
pool
f = 2
s = 2
FC
ŷ
32 x 32 x 1
28 x 28 x 6
14 x 14 x 6
10 x 10 x 16
5 x 5 x 16
120
84
10

# By doing Convolution operation, a feature map is created

$$\frac{n+2p-f}{s}+1 \quad * \quad \frac{n+2p-f}{s}+1$$

| | | | | |
|---|---|---|---|---|
| -1 | 1 | 1 | 1 | -1 |
| -1 | 1 | -1 | 1 | -1 |
| -1 | 1 | 1 | 1 | -1 |
| -1 | -1 | -1 | 1 | -1 |
| -1 | -1 | -1 | 1 | -1 |
| -1 | -1 | 1 | -1 | -1 |
| -1 | 1 | -1 | -1 | -1 |

7*5

*

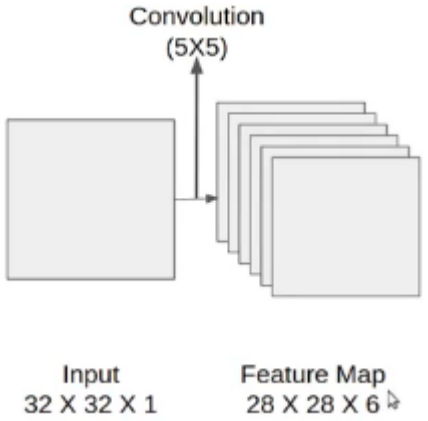
| | | |
|---|---|---|
| 1 | 1 | 1 |
| 1 | -1 | 1 |
| 1 | 1 | 1 |

| | | |
|---|---|---|
| -0.11 | 1 | -0.11 |
| -0.55 | 0.11 | -0.33 |
| -0.33 | 0.33 | -0.33 |
| -0.22 | -0.11 | -0.22 |
| -0.33 | -0.33 | -0.33 |

Feature Map

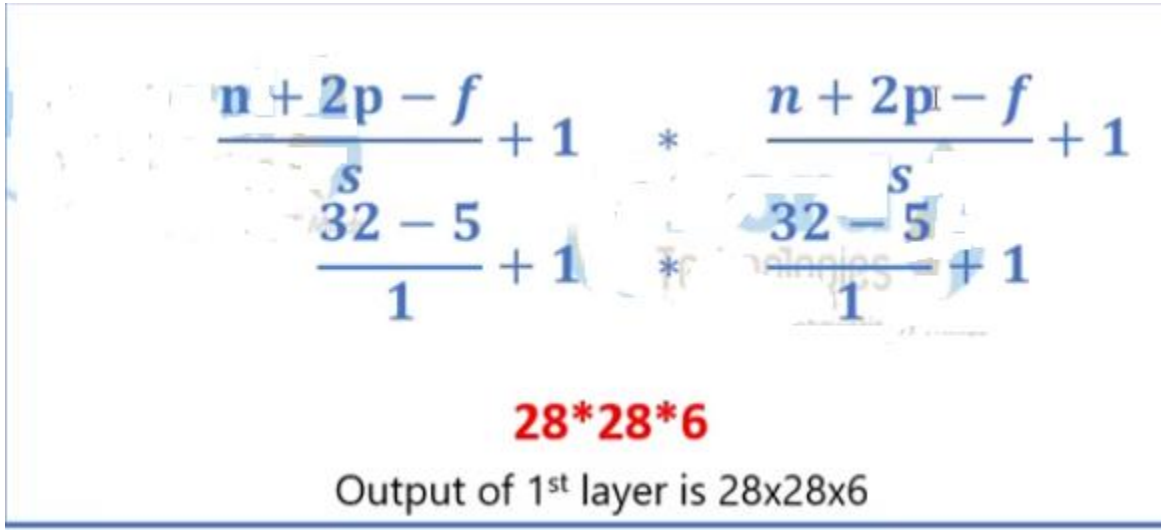
5*3

- The input layer supports an image with dimension 32x32x1, and is passed to the Convolution layer.
- **Layer 1-** is a convolution layer with six 5 × 5 filters. The first convolution operation is applied with the filter of size 5x5 and we have 6 such filters. As a result, we get a feature map of size 28x28x6. Here the number of channels is equal to the number of filters applied.

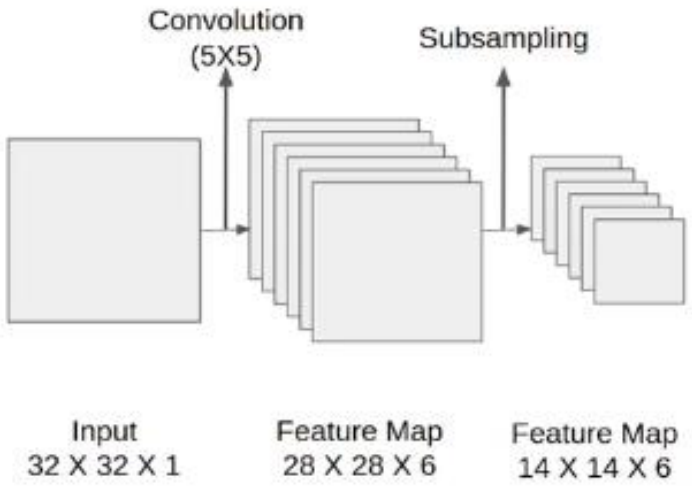



Output shape = ((32-5+1) X (32-5+1) X 6)
= (28 X 28 X 6)

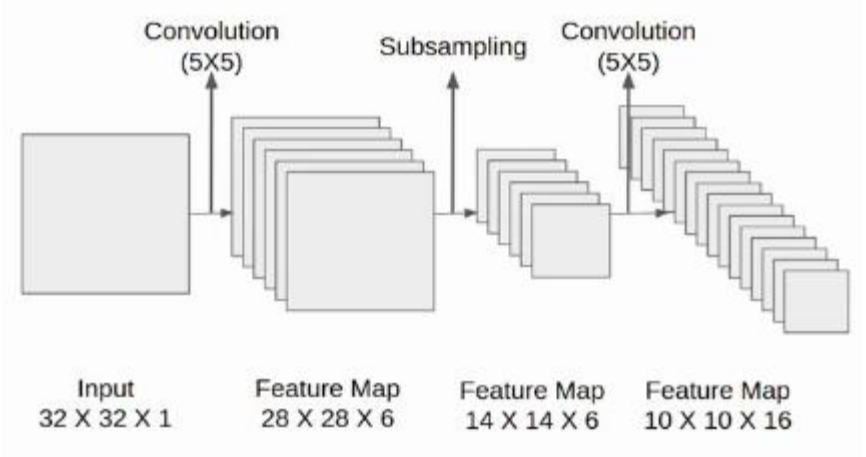
# Input size =32x32x1  6 filters of size 5x5 stride=1

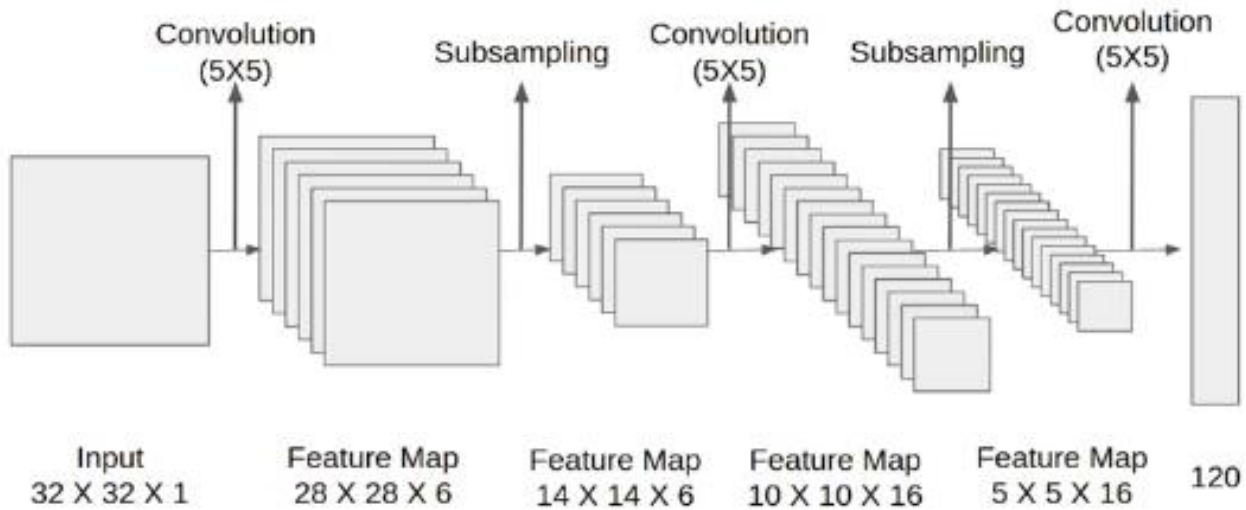


- After the first convolution operation, we apply the average pooling and the size of the feature map is reduced by half. Note that, the number of channels is intact.

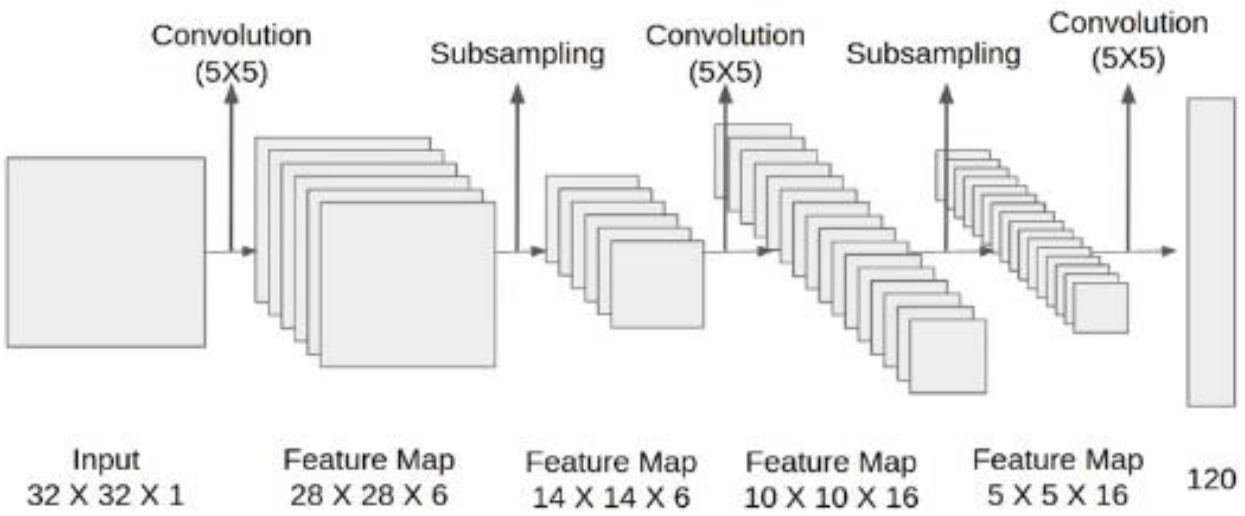


- The second convolution layer encompass **16** 5x5 convolution kernels or filters.
- It returns  a feature map of dimension  10x10x16.
- After this, we again apply an average pooling  which again reduce the size of the feature map by half i.e 5x5x16.



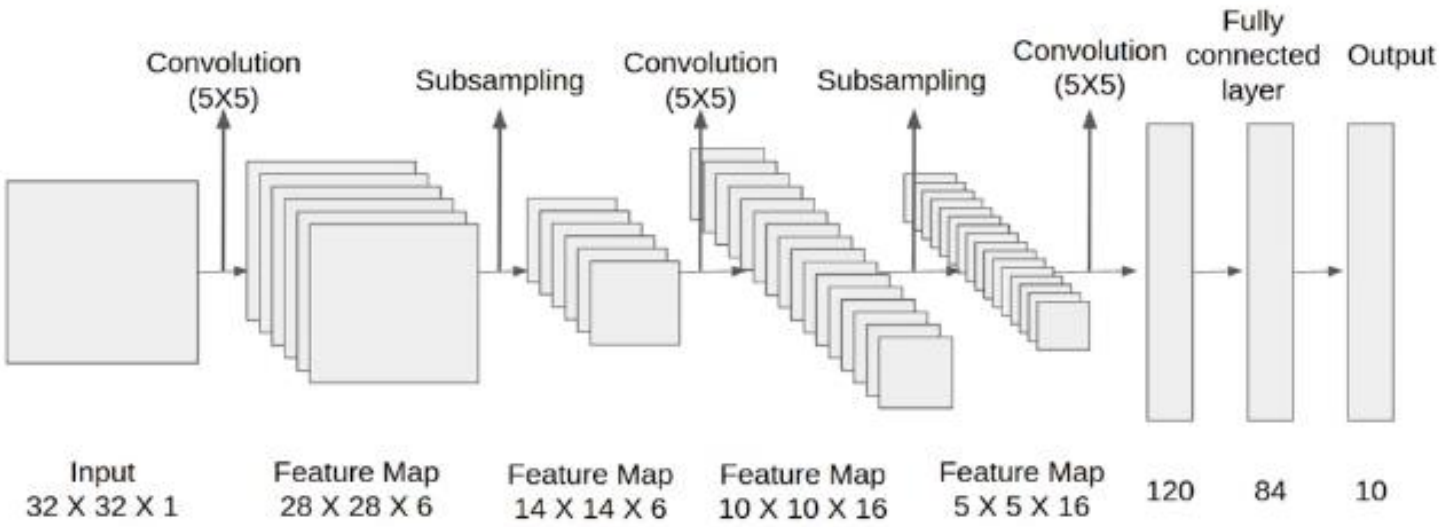

Convolution
(5X5)
Subsampling
Convolution
(5X5)
Subsampling
Convolution
(5X5)
Input
32 X 32 X 1
Feature Map
28 X 28 X 6
Feature Map
14 X 14 X 6
Feature Map
10 X 10 X 16
Feature Map
5 X 5 X 16
120

Then there is a fully connected  with 120 neurons.



After this layer, we have a fully connected layer with 84 neurons. At last, we have an output layer with ten neurons since the data have ten classes.

- Here is the final architecture of the Lenet-5 model.



# LeNet-5 Architecture

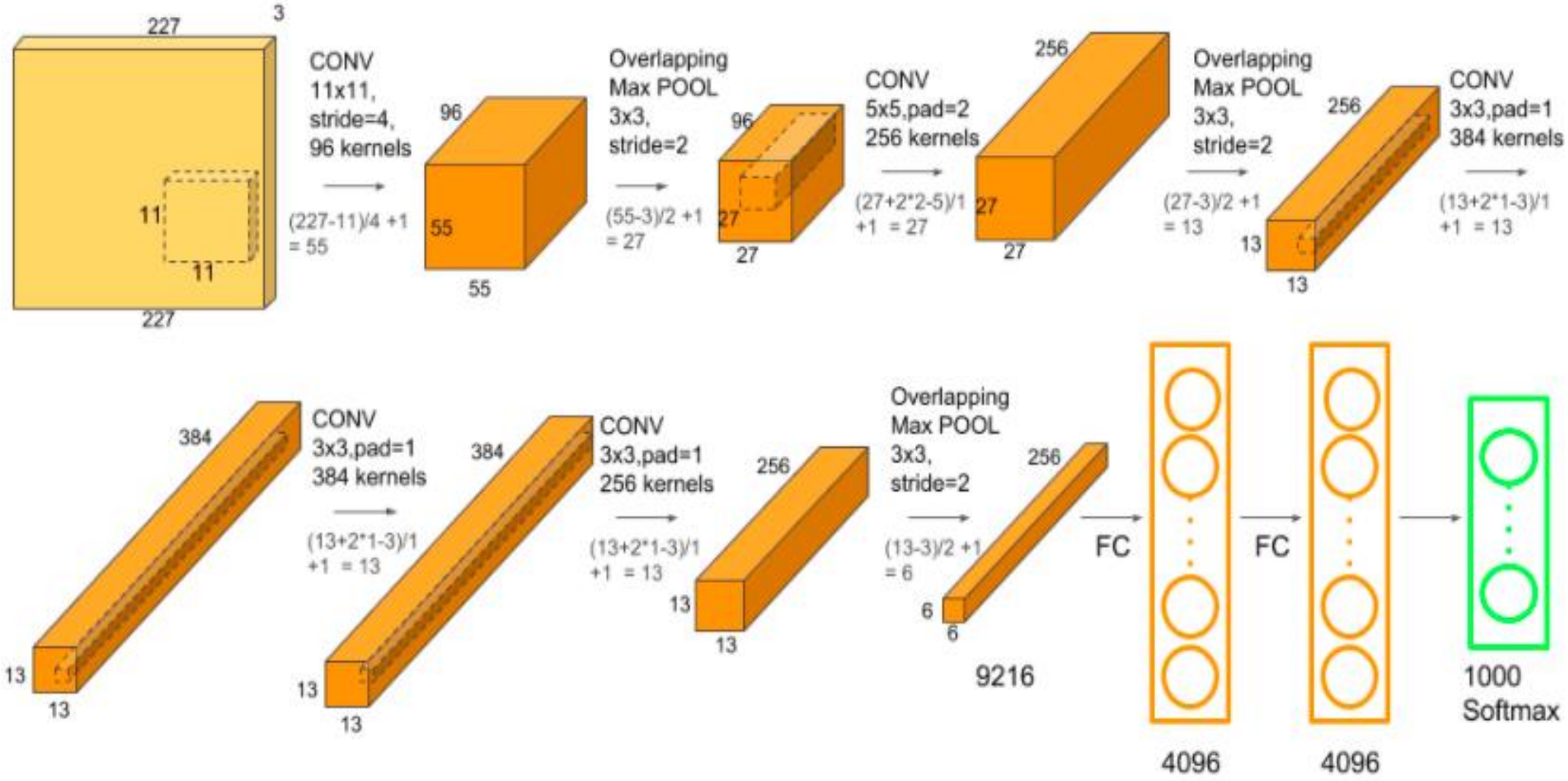
- Architecture Details

| Layer | # filters / neurons | Filter size | Stride | Size of feature map | Activation function |
|---|---|---|---|---|---|
| Input | - | - | - | 32 X 32 X 1 | |
| Conv 1 | 6 | 5 * 5 | 1 | 28 X 28 X 6 | tanh |
| Avg. pooling 1 | | 2 * 2 | 2 | 14 X 14 X 6 | |
| Conv 2 | 16 | 5 * 5 | 1 | 10 X 10 X 16 | tanh |
| Avg. pooling 2 | | 2 * 2 | 2 | 5 X 5 X 16 | |
| Conv 3 | 120 | 5 * 5 | 1 | 120 | tanh |
| Fully Connected 1 | - | - | - | 84 | tanh |
| Fully Connected 2 | - | - | - | 10 | Softmax |

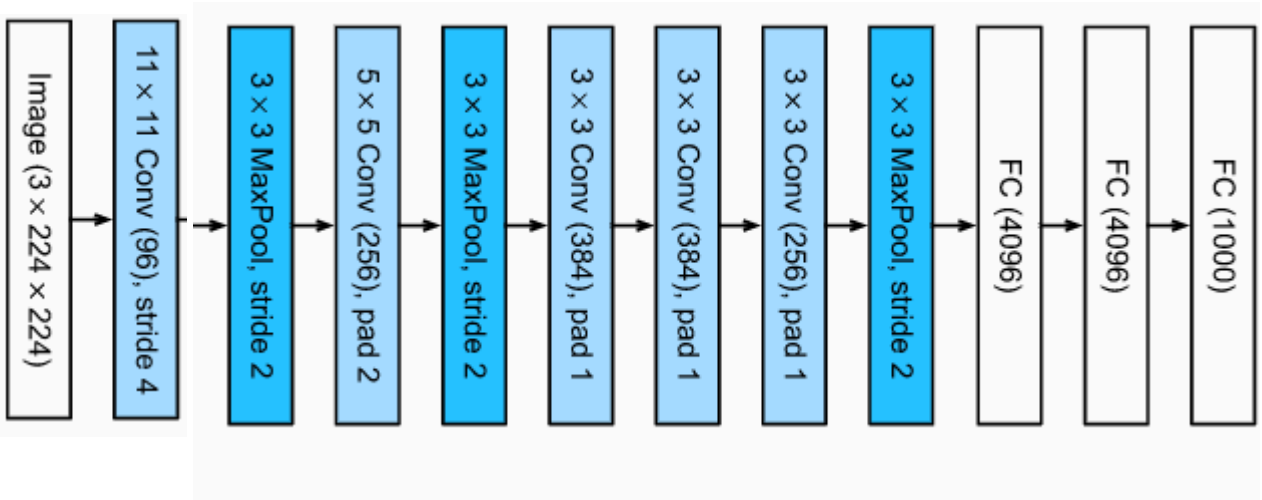
# Alexnet Architecture

- Alex Krizhevsky proposed the AlexNet – a CNN-based network implemented in 2012.
- The input to the Model is RGB images.
- It has 5 convolution layers with a combination of max-pooling layers.
- Then it has 3 fully connected layers.
- The activation function used in all these layers is Relu.
- The activation function used in the output layer is Softmax.

# Alexnet Architecture



# Alexnet Architecture



# Alexnet Architecture

| Layer | # filters / neurons | Filter size | Stride | Padding | Size of feature map | Activation function |
|---|---|---|---|---|---|---|
| Input | - | - | - | - | 227 x 227 x 3 | - |
| Conv 1 | 96 | 11 x 11 | 4 | - | 55 x 55 x 96 | ReLU |
| Max Pool 1 | - | 3 x 3 | 2 | - | 27 x 27 x 96 | - |
| Conv 2 | 256 | 5 x 5 | 1 | 2 | 27 x 27 x 256 | ReLU |
| Max Pool 2 | - | 3 x 3 | 2 | - | 13 x 13 x 256 | - |
| Conv 3 | 384 | 3 x 3 | 1 | 1 | 13 x 13 x 384 | ReLU |
| Conv 4 | 384 | 3 x 3 | 1 | 1 | 13 x 13 x 384 | ReLU |
| Conv 5 | 256 | 3 x 3 | 1 | 1 | 13 x 13 x 256 | ReLU |
| Max Pool 3 | - | 3 x 3 | 2 | - | 6 x 6 x 256 | - |
| Dropout 1 | rate = 0.5 | - | - | - | 6 x 6 x 256 | - |

# Alexnet Architecture

- The input to this model is the images of size 227x227x3.

**Convolution and Maxpooling Layers**

First we apply the first convolution layer with 96 filters of size 11x11 with stride 4. The activation function used in this layer is relu. The output feature map is 55x55x96.

Also, the number of filters becomes the channel in the output feature map.

- Next, we have the first Maxpooling layer, of size 3X3 and stride 2. Then we get the resulting feature map with the size 27x27x96.

# Alexnet Architecture

- After this, we apply the second convolution operation.
- This time the filter size is reduced to 5X5 and we have 256 such filters. The stride is 1 and padding 2.
- The activation function used is again relu. Now the output size we get is 27x27x256.
- Again there is a max-pooling layer of size 3X3 with stride 2. The resulting feature map is of shape 13x13x256.

# Alexnet Architecture

- Now we apply the third convolution operation with 384 filters of size 3X3 stride 1 and also padding 1. Again the activation function used is relu. The output feature map is of shape 13X13X384.
- Then we have the fourth convolution operation with 384 filters of size 3X3. The stride along with the padding is 1. On top of that activation function used is relu. Now the output size remains unchanged i.e 13X13X384.
- After this, we have the final convolution layer of size  3X3 with 256 such filters. The stride and padding are set to one also the activation function is relu. The resulting feature map is of shape 13X13X256.

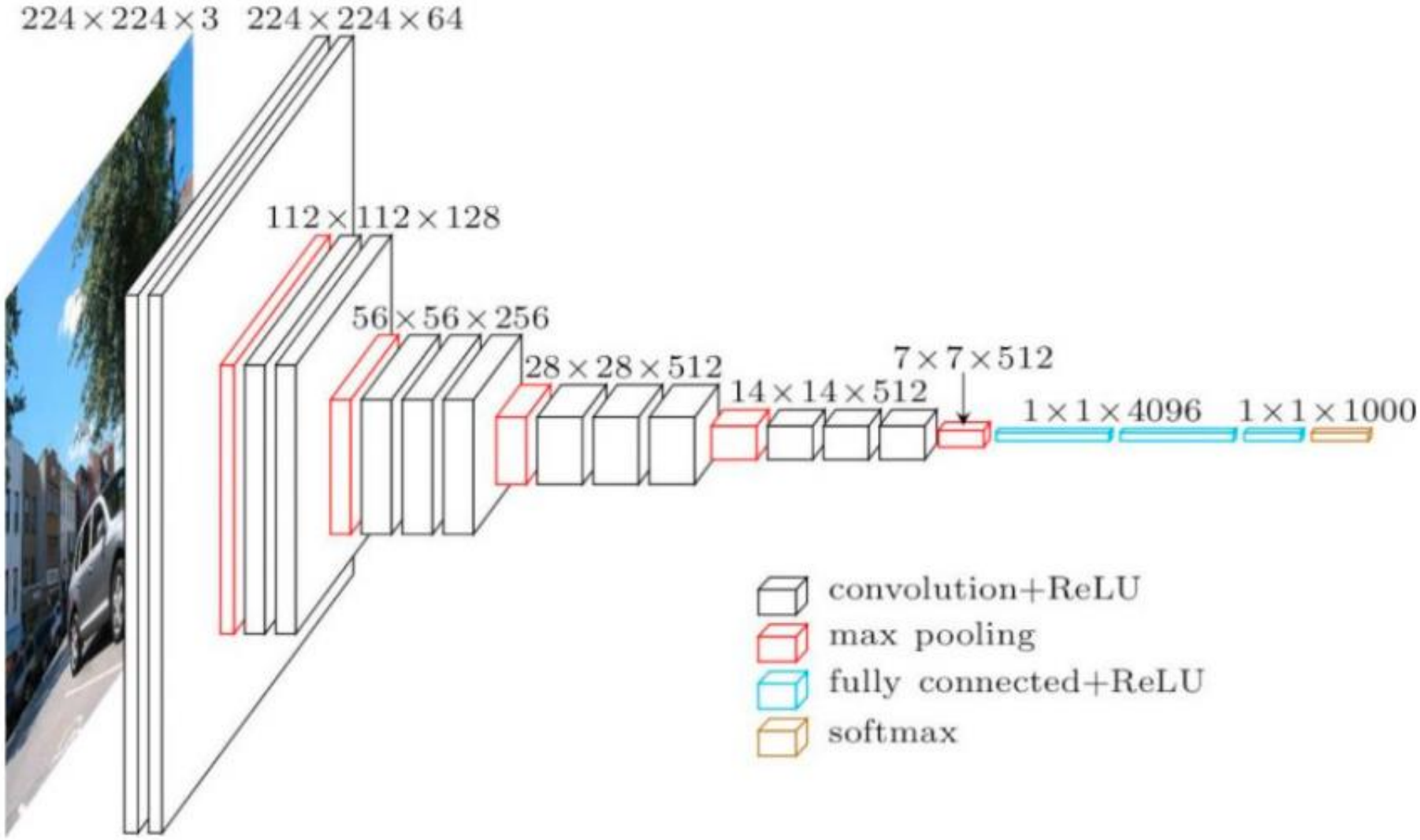
# Alexnet Architecture

- So if you look at the architecture, the number of filters is increasing as we are going deeper.  Hence it is extracting more features as we move deeper into the architecture.
- Also, the filter size is reducing, which means the initial filter was larger and as we go ahead the filter size is decreasing, resulting in a decrease in the feature map shape.Next, we apply the third max-pooling layer of size 3X3 and stride 2. Resulting in the feature map of the shape 6X6X256.

# VGG-16 (Visual Geometry Group -16)

- VGG16 is a simple and widely used Convolutional Neural Network (CNN) Architecture used for ImageNet - dataset of over 14 million images belonging to 1000 classes, a large visual database project used in visual object recognition software research.
- The VGG16 Architecture was developed in the year 2014.
- VGG16 is used in many deep learning image classification techniques and is popular due to its ease of implementation.

# VGG-16 (Visual Geometry Group -16)



- The input to any of the network configurations is considered to be a fixed size 224 x 224 image with three channels – R, G, and B.
- The only pre-processing done is normalizing the RGB values for every pixel.
- This is achieved by subtracting the mean value from every pixel.
- Image is passed through the first stack of 2 convolution layers of the very small receptive size of 3 x 3, followed by ReLU activations. Each of these two layers contains 64 filters.
- The convolution stride is fixed at 1 pixel, and the padding is 1 pixel.
- This configuration preserves the spatial resolution, and the size of the output activation map is the same as the input image dimensions.

- The activation maps are then passed through spatial max pooling over a 2 x 2-pixel window, with a stride of 2 pixels.
- This halves the size of the activations. Thus the size of the activations at the end of the first stack is 112 x 112 x 64.
- The activations then flow through a similar second stack, but with 128 filters as against 64 in the first one. Consequently, the size after the second stack becomes 56 x 56 x 128.
- This is followed by the third stack with three convolutional layers and a max pool layer.
    - The no. of filters applied here are 256, making the output size of the stack 28 x 28 x 256.
- This is followed by two stacks of three convolutional layers, with each containing 512 filters.
    - The output at the end of both these stacks will be 7 x 7 x 512.

- The stacks of convolutional layers are followed by three fully connected layers with a flattening layer in-between.
- The first two have 4,096 neurons each, and the last fully connected layer serves as the output layer and has 1,000 neurons corresponding to the 1,000 possible classes for the ImageNet dataset.
- The output layer is followed by the Softmax activation layer used for categorical classification.

# VGG-16 Architecture Summary

- 2 x convolution layer of 64 channel of 3x3 kernal and same padding
- 1 x maxpool layer of 2x2 pool size and stride 2x2
- 2 x convolution layer of 128 channel of 3x3 kernal and same padding
- 1 x maxpool layer of 2x2 pool size and stride 2x2
- 3 x convolution layer of 256 channel of 3x3 kernal and same padding
- 1 x maxpool layer of 2x2 pool size and stride 2x2
- 3 x convolution layer of 512 channel of 3x3 kernal and same padding
- 1 x maxpool layer of 2x2 pool size and stride 2x2
- 3 x convolution layer of 512 channel of 3x3 kernal and same padding
- 1 x maxpool layer of 2x2 pool size and stride 2x2

Also add relu(Rectified Linear Unit) activation to each layers so that all the negative values are not passed to the next layer.

2-1-2-1-3-1-3-1-3-1-FC-FC-FC

# Cifar_10 dataset

